OIM6301 - PROGRAMMING FOR BUSINESS ANALYTICS

OIM6301 - Database Project

Overview

You will design, build, and analyze a small relational database **from scratch** using **SQLite in DBeaver**, with data generated by **Mockaroo**. You are allowed to use AI tools (ChatGPT, Copilot, etc.) to support ideation and troubleshooting, but you must document and reflect on your usage.

What you will do:

1. **Design** an ERD and draft your schema
2. **Generate** CSV files with Mockaroo
3. **Build** a SQLite database in DBeaver and import your CSV files
4. **Query** the database to answer business questions, including some you design yourself

You will select **one** detailed scenario below. You may propose your own with instructor approval if it is of similar scope and complexity.

Scenarios (choose one)

Each scenario is written as if a real Babson-affiliated organization is moving from paper/legacy tools to a database system. Below, we list **Data Currently Collected** to guide your modeling. You may use **more or fewer** fields than listed, and you do **not** have to use every item. We intentionally do **not** specify IDs or keys: you must decide what tables, keys, and relationships make sense.

Scenario A - BabsonAI Labs (Campus AI Startup)

BabsonAI Labs is an early-stage startup founded by Babson students. Until now, project tracking has been done in spreadsheets, emails, and chat threads, making it hard to see progress, budgets, and workload. They want a simple relational database to support operational reporting.

Data currently collected (just examples; you may include more/less as appropriate):

- People working at the startup: first name, last name, role/title, work email, start date
- Projects: project name, category (e.g., LLM app, data pipeline, internal tooling), budget, start date, status text
- Tasks and activities per project: task name/description, assigned person, due date, completed yes/no, completion date
- Project expenses: expense date, amount, expense category (e.g., GPU cloud, data labeling, SaaS), short memo/notes
- Optional operational context you may include: priority levels, client name, project stage, sprint name, time tracked (hours), attachment/URL references

Purpose of digitization: a lightweight backbone to answer questions like “Where are we spending?”, “Which projects are at risk?”, and “Who is overloaded?”

Scenario B - Babson Investment Club (BIC) (Student Portfolios, Stocks & Crypto)

BIC keeps informal records across Google Sheets and broker screenshots. They want to consolidate membership, portfolios, assets, and trades in a database to produce regular reports and educate new members.

Data currently collected (just examples; you may include more/less as appropriate):

- Club members: first name, last name, major, join date, school email
- Portfolios owned by members: portfolio name, created date, optional notes/strategy label
- Tradable assets tracked by the club: ticker/symbol, asset type (stock, crypto, fund), current price snapshot, market/venue
- Transactions within portfolios: trade date, trade type (buy/sell), quantity/units, trade price; optional fee notes

- Optional educational context: sector/category for assets, risk rating, research link, mentor assigned, meeting date

Purpose of digitization: produce a clear audit trail of trades and portfolios, run monthly summaries, and support learning goals (e.g., diversification, position sizing, P&L approximations).

Scenario C - Babson Bean Café (Campus Café & Loyalty)

The campus café uses a POS plus a punch-card loyalty program; data is fragmented and not queryable. A small database should unify products, orders, and customer loyalty to support sales analysis and promotions.

Data currently collected (just examples; you may include more/less as appropriate):

- Customers: first name, last name, email, join date, accumulated loyalty points
- Products: product name, category (drink/food/other), price, cost, availability flag, optional size/variant
- Orders: order date/time, customer reference, payment method (cash/card/mobile), optional order channel (in-store/app)
- Order line items: product reference, quantity, optional customizations/notes
- Optional promotion context: coupon code, points redeemed on order, campaign name, barista name

Purpose of digitization: unify sales and loyalty to identify best-sellers, high-value customers, margin trends, and targeted offers.

Scenario D - Babson PT Clinic (Physical Therapy)

A local physical therapy clinic affiliated with Babson alumni wants to move away from paper charts. They need a basic data backbone for scheduling, visit documentation, and procedure analysis.

Data currently collected (just examples; you may include more/less as appropriate):

- Caregivers/therapists: first name, last name, degree/credential, phone, email
- Patients: first name, last name, date of birth, address, city, state, zip, phone, email

- Therapy/procedure catalog: description, price (charged), cost (internal)
- Visits: visit date, caregiver reference, patient reference, procedures/services performed
- Optional operational context: diagnostic notes, therapy duration, payment method, insurance indicator, follow-up recommended

Purpose of digitization: improve accuracy/efficiency, track revenue and margins by therapy, and identify patients due for follow-up.

Phase 1 - Database Design (ERD)

1. Draw an ERD for your chosen scenario with at least 3–4 entities.
2. Identify relationships (1-to-many, many-to-many). If you include a many-to-many, model the junction entity explicitly.
3. Decide your attributes, constraints, and naming conventions. Keep types reasonable (text, number/decimal, date/time, boolean).
4. Do not copy a template ERD. Start from the "Data currently collected" lists and your business reasoning.
5. Export your ERD to an image (e.g., `ERD.png`) for submission.

📦 **Deliverables for this step:**

- `ERD.png` - Your entity-relationship diagram image
- Short paragraph in `README.md` summarizing your entities and relationships

Phase 2 - Data Generation

You will fabricate realistic sample data for each entity using Mockaroo.

1. Build one Mockaroo schema per entity in your ERD. Align field names with your design. (No need to pay for Mockaroo.)
2. Recommended minimum volume: 20–50 rows per entity, but you can adjust up or down if justified (Mockaroo lets you generate up to 1,000 rows of realistic test data in CSV, JSON, SQL, and Excel formats.).

3. Field suggestions:
    - Names and emails: use Mockaroo's built-in First Name, Last Name, Email
    - Money: Number (decimal, 2 places) or Price
    - Dates: Date/Datetime; restrict ranges to match scenario windows
    - Boolean: True/False
    - Categories: use Enum values (e.g., asset type, payment method, project category)
4. Generate parent entities first (those referenced by others), then generate child entities. For foreign keys in CSVs, use a number range matching the parent set you produced.
5. Export each dataset to a separate UTF-8 CSV file. Keep a consistent naming convention (e.g., `Employees.csv` , `Projects.csv` , ...).

Tutorials:

- How to generate datasets related by a foreign key using Mockaroo
- How to use formulas to create custom data types in Mockaroo
- You may also ask AI to help plan your schemas, enum values, and ranges; you must still validate they work.

📦 **Deliverables for this step:**

- `/seed/` folder containing all your CSV files (one per entity)
- Brief notes in `README.md` describing any assumptions or data generation decisions

Phase 3 - Database Implementation (SQLite & DBeaver)

3.1 Create a new SQLite database file

1. Open DBeaver → Database → New Database Connection → SQLite
2. Click "Create" button and enter a file path (e.g., `C:\data\midterm_<yourname>.db`) and click "Test Connection" to verify.

3. Click “Finish” to create the connection.

3.2 Create tables with DDL

1. Open a new SQL Editor on your SQLite connection (right-click on your database → SQL Editor → New SQL Script)

2. Enable foreign-key enforcement for your session (add this at the top of your `schema.sql`):

```sql
PRAGMA foreign_keys = ON;
```

3. Write `CREATE TABLE` statements for every entity in your ERD. Include primary/unique keys, reasonable types, NOT NULL where appropriate, and foreign-key constraints for relationships.

   Example DDL for Scenario A (BabsonAI Labs):

```sql
-- Example: People table
CREATE TABLE People (
    person_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    role TEXT,
    work_email TEXT UNIQUE NOT NULL,
    start_date TEXT  -- SQLite stores dates as TEXT in format 'YYYY-MM-DD'
);

-- Example: Projects table
CREATE TABLE Projects (
    project_id INTEGER PRIMARY KEY AUTOINCREMENT,
    project_name TEXT NOT NULL,
    category TEXT,
    budget REAL,  -- Use REAL for decimal/money values
    start_date TEXT,
    status TEXT
```

```sql
);

-- Example: Expenses table with foreign key
CREATE TABLE Expenses (
    expense_id INTEGER PRIMARY KEY AUTOINCREMENT,
    project_id INTEGER NOT NULL,
    expense_date TEXT NOT NULL,
    amount REAL NOT NULL,
    expense_category TEXT,
    notes TEXT,
    FOREIGN KEY (project_id) REFERENCES Projects(project_id)
);
```

Important Notes:

- **DBeaver does not auto-generate DDL from diagrams.** You must write your CREATE TABLE statements manually.
- **You are encouraged to use AI tools** (ChatGPT, Claude, Copilot) to generate your `schema.sql` based on your ERD, BUT you must:
- Carefully review every line of generated SQL
- Verify data types match your CSV files
- Ensure foreign key references are correct
- Test by executing the DDL before importing data
- **Common SQLite data types:**
- `INTEGER` - whole numbers, use with `PRIMARY KEY AUTOINCREMENT`
- `REAL` - floating-point numbers (for money, decimals)
- `TEXT` - strings, dates, emails, etc.
- `BLOB` - binary data (rarely used in this project)

References:

- SQLite Data Types

    - SQLite Foreign Key Support
    - W3Schools SQLite Tutorial
4. Execute your DDL by clicking the "Execute SQL Statement" button (or press `Ctrl+Enter`)
5. Verify the tables exist: expand your database in the Database Navigator and refresh to see your new tables

3.3 Import CSV data

Step-by-step CSV import process:

1. **Prepare for import**: Ensure your CSV files are saved in UTF-8 encoding and your tables are created successfully
2. **Start import wizard**:
    - In Database Navigator, expand your database and locate your target table
    - Right-click on the table → **Import Data**
    - Select "CSV" as the input format → Click **Next**
3. **Select source file**:
    - Click **Browse** and select your CSV file (e.g., `Projects.csv`)
    - Click **Next**
4. **Configure import settings** (Important!):
    - **Header**: Check "First line is header" if your CSV has column names in the first row
    - **Delimiter**: Usually comma (`,`), but verify it matches your CSV
    - **Encoding**: Select **UTF-8**
    - Click **Next**
5. **Map columns**:
    - DBeaver will try to auto-map CSV columns to table columns by name
    - Verify each mapping is correct (Source Column → Target Column)
    - **Important**: If you have an `AUTOINCREMENT` primary key (e.g., `project_id`), you can either:
        - Uncheck that column to let SQLite auto-generate IDs, OR

        - Include specific IDs in your CSV and map them
    - Click **Next**
6. **Review and start**:
    - Review the import summary
    - Click **Start** to begin import
    - Wait for completion message
7. **Verify import**:
    - Right-click on the table → **View Data** (or press F3)
    - Run: `SELECT COUNT(*) FROM Projects;` to verify row count

Import order matters!

- Import parent tables first (tables with no foreign keys), then child tables
- Example order for Scenario A:
    i. People (no foreign keys)
    ii. Projects (no foreign keys)
    iii. Tasks (references People and Projects)
    iv. Expenses (references Projects)

Troubleshooting common import issues:

- "**Foreign key constraint failed**": You're trying to import a child table before its parent. Import the parent first.
- "**Date format error**": Your CSV date format doesn't match SQLite expectations. Use `YYYY-MM-DD` format.
- "**Column count mismatch**": Your CSV has more/fewer columns than your table. Check your column mapping.
- "**Encoding issues (weird characters)**": Re-save your CSV with UTF-8 encoding (in Excel: Save As → CSV UTF-8)

Tutorial documentation: DBeaver Data Transfer Guide

3.4 Perform quick data quality checks

After importing all your data, verify data integrity. **These checks are optional but recommended examples -** choose the ones relevant to your schema.

Visual checks in DBeaver (quick and easy):

1. **View table schema**: Right-click on table → View Diagram (or Properties) to verify structure
2. **Browse data**: Right-click on table → View Data (or press F3) to spot-check rows
3. **Check row counts**: In Database Navigator, table names show row counts in parentheses
4. **Verify relationships**: Expand table → Foreign Keys to see defined relationships

SQL-based checks (examples you can adapt):

Check for orphan foreign keys (should return zero rows):

```
-- Example: Verify all expenses reference valid projects
SELECT e.expense_id, e.project_id
FROM Expenses e
LEFT JOIN Projects p ON e.project_id = p.project_id
WHERE p.project_id IS NULL;
```

Verify row counts:

```
SELECT COUNT(*) FROM People;   -- Should match your CSV row count
SELECT COUNT(*) FROM Projects;
```

Check for unexpected NULLs or data issues:

```
-- Example: Check for missing required data
SELECT * FROM People WHERE work_email IS NULL;
```

```sql
-- Example: Check for unrealistic values
SELECT * FROM Expenses WHERE amount < 0;
```

📦 **Deliverables for this step:**

- `schema.sql` - All CREATE TABLE statements with `PRAGMA foreign_keys = ON;` at the top
- Screenshots of successful imports and row counts (can be in `/evidence/` folder or embedded in your documentation)

Phase 4 - Business Analysis (SQL Queries)

IMPORTANT: Every query in your `queries.sql` file MUST begin with a comment block that includes:

1. The business question being answered
2. How the output would be used in practice

You must submit **at least 8 queries** total:

- **6 required** addressing listed business questions for your chosen scenario (adapt them to your model)
- **2 original queries** that you design yourself to answer meaningful new questions

Note on originality: Your two original queries must not repeat the logic of the provided examples. They should demonstrate curiosity, insight, or creativity in exploring your data. Think about what questions a real business stakeholder would want answered.

Format all SQL code neatly and use logical column aliases.

Recommended variety across your 8 queries (demonstrates SQL proficiency from basic to advanced):

- Simple `SELECT` with filtering (`WHERE` clause)
- Aggregation queries (`COUNT` , `SUM` , `AVG` , `MIN` , `MAX`)
- Multi-table `JOIN` queries (combine data from 2+ tables)

- `GROUP BY` with `HAVING` (filtering grouped results)
- Subquery or CTE (Common Table Expression)
- **Advanced (optional)**: Date analysis, window functions, or ranking (e.g., `ROW_NUMBER()`, `RANK()`, monthly trends, top-N)

Save everything in `queries.sql` and include key output screenshots in `/evidence/`.

Below are example business questions per scenario. **These are suggestions for reference only.** If your database design and data don't support a particular query, or if you can think of more meaningful queries for your scenario, feel free to write your own queries that make business sense for your design.

Scenario A - BabsonAI Labs

- Show each project's total expenses versus budget and the variance percentage, sorted by largest over-spend
- For each person, count completed vs pending tasks in the last 30 days
- Summarize projects by status and compute average task completion rate per status
- List overdue tasks with assigned person and project name
- Aggregate expenses by expense category; return only categories with total spend above the overall average (HAVING)
- Identify the top 3 most active projects by completed tasks in the last 90 days (you may use a subquery or a window function if supported)
- Two original queries of your own design

Scenario B - Babson Investment Club (BIC)

- For each portfolio, count trades and sum traded value in the last 30 days (separate buy and sell if desired)
- By asset type (stock/crypto/fund), show trade counts and share of total traded value
- For each member, compute total buy value and total sell value; estimate gross profit (simplified)
- Find the top 5 assets by approximate current holdings across all portfolios (e.g., buys − sells)
- Show monthly trading volume and net buy amount by year-month
- Compare average portfolio cost basis on a symbol to the symbol's current price (approximate unrealized gain/loss)

- Two original queries of your own design

Scenario C - Babson Bean Café

- Find the top 3 best-selling products in the last month by quantity
- For each customer, compute total spend, order count, and average ticket value; filter to those above average ticket value
- By product category, compute revenue, cost, gross margin, and margin percentage
- Identify customers with high points but no orders in the last 60 days (re-engagement list)
- Show distribution of orders by payment method
- Produce a monthly sales trend and identify the highest-revenue month
- Two original queries of your own design

Scenario D - Babson PT Clinic

- List patients with no visits (data cleaning)
- For each patient, show the most recent visit date with contact information
- For a given date, show each therapy performed and how many times
- Identify patients not seen in the last 90–180 days (outreach list)
- For each caregiver, show count of procedures overseen, average revenue, and total revenue; order by total revenue desc
- For each therapy, compute total revenue, total cost, gross margin, and margin percentage; order by margin desc
- Two original queries of your own design

📦 **Deliverables for this step:**

- `queries.sql` - All 8+ queries with detailed comments (business question, explanation, SQL code)
- Screenshots of key query outputs (can be in `/evidence/` folder or embedded in your documentation)

Use of AI (Allowed and Encouraged) and Required Documentation

You may use AI tools to:

- Brainstorm entities, attributes, and relationships from the scenario text
- Draft DDL or example queries
- Plan Mockaroo schemas and enum values
- Troubleshoot SQL errors and refine queries

Requirements:

Write `reflection.md` (300–400 words) addressing:

- What you learned about relational modeling and SQL from end to end
- How AI supported your work and where it fell short
- **AI Usage Documentation**: Include 3-5 key prompts you used with brief notes on how you validated or modified the output
- At least two concrete AI errors or misleading suggestions you corrected
- What you would change in your approach next time

What to Submit

Submission Structure (Simplified)

You will submit **4 main components**:

1. `project-documentation.md` (or `.docx`) - Your main documentation file containing:
    - Scenario choice and rationale
    - ERD image (embedded or attached)

    - Database design assumptions and decisions
    - Brief notes on data generation process
    - Steps you followed (summary of your process)
    - Screenshots of successful imports and row counts
2. `reflection.md` (or `.docx`) - Your reflection document (300–400 words) containing:
    - What you learned about relational modeling and SQL from end to end
    - How AI supported your work and where it fell short
    - **AI Usage Documentation**: 3-5 key prompts you used with brief notes on how you validated or modified the output
    - At least two concrete AI errors or misleading suggestions you corrected
    - What you would change in your approach next time
3. `schema.sql` - All CREATE TABLE statements (SQLite syntax) with `PRAGMA foreign_keys = ON;` at the top
4. `queries.sql` - All 8+ queries with detailed comments. Each query MUST include:
    - Business question being answered
    - Brief explanation of how the output would be used
    - The SQL query itself with proper formatting

Optional but recommended:

- `/seed/` folder with your CSV files (helpful for reproducibility)
- `/evidence/` folder with additional screenshots if you have many query outputs or want to organize them separately (you can also embed screenshots directly in your `project-documentation.md`)

Submission Format Options

Option A: Markdown files (recommended for GitHub users)

- Keep all files as `.md` and `.sql` as shown above

- See “Submission Method 2: GitHub” below
- **New to Markdown?** Learn the basics:
  - Markdown Guide - Basic Syntax
  - GitHub Markdown Guide

Option B: Word documents (if you’re not comfortable with Markdown)

- Create your documentation directly in Word (`.docx` format)
- Document 1: Project Documentation (equivalent content to `project-documentation.md`)
- Document 2: Reflection (equivalent content to `reflection.md`)
- Keep `.sql` files as plain text files (not in Word)

Submission Methods

Method 1: Canvas Submission (Required for grading)

- Compress ALL files into a single `.zip` file named: `LastName_FirstName_DBProject.zip`
- Upload the `.zip` file to Canvas before the deadline
- **Important**: Canvas requires a `.zip` file for submissions with multiple files

Method 2: GitHub Submission (Optional, but encouraged)

If you are comfortable with Git and GitHub:

1. In your personal `OIM6301` repository, create a folder: `proj-db/`
2. Place all your files in this folder
3. Commit and push to GitHub
4. Still submit the `.zip` file to Canvas (this is required for grading)
5. Optionally include your GitHub repository link in your documentation

Example GitHub structure:

```
OIM6301/
  proj-db/
    project-documentation.md
    reflection.md
    schema.sql
    queries.sql
    seed/
      People.csv
      Projects.csv
      ...
    evidence/
      import-success.png
      query-outputs.png
      ...
```

Scoring Rubric (100 points)

- ERD & Schema Design (20 points)
- Data Generation & Import (20 points)
- SQL Queries: correctness, variety, clarity, originality (30 points)
- Reflection and AI Usage Documentation: insightfulness, learning evidence, prompts validation (30 points)

Notes:

- Creativity in your original queries matters. Make them meaningful for decision-making in your scenario.
- Consistency and data quality count. Show quick checks and fixes if you discover issues.

Practical Tips

- Import parent entities before child entities.
- Keep data types consistent between CSVs and DDL; test a small sample first.
- Use comments in `queries.sql` to explain the business question each query answers.
- Enable foreign keys at the top of every SQLite session: `PRAGMA foreign_keys = ON;` .
- Save evidence as you go to avoid last-minute screenshots.

Common Issues and Solutions

SQLite Date Handling

- SQLite stores dates as TEXT. Use format: `YYYY-MM-DD` or `YYYY-MM-DD HH:MM:SS`
- In queries, use date functions: `date()` , `datetime()` , `julianday()` , `strftime()`
- Example: `WHERE date(order_date) >= date('2024-01-01')`

CSV Import Encoding

- Ensure UTF-8 encoding without BOM (Byte Order Mark)
- If special characters appear garbled, re-export CSV with UTF-8 encoding
- In Excel: Save As → CSV UTF-8 (Comma delimited)

Foreign Key Constraint Violations

- Import order matters: parents before children
- Verify parent IDs exist before importing child tables
- Use the orphan check query provided in Phase 3.4 to validate relationships

Mockaroo-to-SQLite Type Mapping

- Mockaroo “Number” → SQLite `INTEGER` or `REAL`
- Mockaroo “Date” → SQLite `TEXT` (date format `YYYY-MM-DD`)
- Mockaroo “Money” → SQLite `REAL`
- Mockaroo “Boolean” → SQLite `INTEGER` (0/1) or `TEXT` (‘true’/’false’)

AUTOINCREMENT Primary Keys

- If using `AUTOINCREMENT` , either:
  - Let SQLite generate IDs automatically (don’t include ID column in CSV), OR
  - Include explicit IDs in your CSV (useful for maintaining foreign key relationships)
- For child tables, make sure foreign key values match the parent IDs you generated

updated: *2024/10/14*